

claude-windows / 7dc3da51-af38-47b6-83f6-fdda83b60664

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-inde  
xer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf_56dfabe6-f8c\agent-  
a85e28a4a4006a191.jsonl`  
- SHA-256: `82db2a0f010d684fd76051fdd951826c0f31f1675f720babfffb3ecece8cafd0a`  
- Source modified: `2026-07-07T00:20:30+00:00`  
- Imported at: `2026-07-08T16:00:05+00:00`  
- project: `wf_56dfabe6-f8c`  
- session_id: `7dc3da51-af38-47b6-83f6-fdda83b60664`
```

Transcript

1. user (2026-07-07T00:17:09.273Z)

Reviewing GitHub PR #57 "F7: add /why explainability command" (branch feat/f7-why-command -> main). ADR 0012 F7 (final phase). +295/-2, 5 files.

Checked-out copy at C:/Users/avidu/Projects/_wt-pr57. Run repros:

PYTHONPATH=C:/Users/avidu/Projects/_wt-pr57/src

C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe <script>.

WHAT F7 DOES:

```
- command_service.py: a /why command (regex allows "/why" alone or "/why <arg>").  
_handle_why(conn, user_id, arg, cfg): if arg is None or  
    not arg.strip().isdecimal() or int(arg) < 1 -> Usage reply. Else item =  
db.get_content_item_for_account(conn, user_id, item_id);  
    if item is None -> "Content item `<id>` was not found for your account."; else decisions =  
db.list_policy_decisions_for_content(item.id);  
    if decisions -> _format_policy_decisions(item, decisions); else ->  
_format_missing_policy_decision(item, current_cfg).  
- db.py get_content_item_for_account: SELECT * FROM content_items WHERE id=? AND account_id=?  
(the account-scoped IDOR guard).  
- _format_policy_decisions renders each decision + stage (reason + metadata via  
_format_json_value = json.dumps(default=str, ensure_ascii=False), each value truncated to 500  
chars).  
- _format_missing_policy_decision: replays the CURRENT FilterConfig via FilterEngine over a  
VideoMeta built from persisted content_item fields  
    (duration/width/height/size/mime only), with a caveat that historical config changes may  
differ.
```

VERIFIED by the main reviewer: gate green (560 passed @ 93.99%); IDOR handled ? account A doing /why on account B item returns "not found for your account"

with NO leak of B channel/caption/url/domain; B sees own trace; malformed (/why, /why abc, /why -1, /why 0) -> Usage. Tests cover cross-account access + scoping.

Ground rules: READ code, cite file:line. Run repros. Report ONLY defects in THIS PR diff. No praise. Concrete failure scenario, ranked.

Severity: high (access-control bypass / cross-account leak, or crash on user input), medium (real edge bug), low (rendering/limit/note).

DIMENSION: input parsing + rendering robustness. isdecimal() unicode digits (Arabic-Indic, full-width, superscripts) may pass isdecimal but int() differs ? crash or surprising item_id? very large id -> big int, no overflow, no match? extra spaces? The reply can be large (many decisions x stages x metadata; each value truncated 500 but the whole message unbounded) -> exceeds the Telegram ~4096 send limit -> _reply catches RPCError so it silently fails ? acceptable or should the whole message be capped? Any markdown/backtick breakout injection from attacker-controlled caption/domain/url echoed into the reply? The metadata-replay fallback is inaccurate for content-type filters (gif/round/nosound not persisted on content_items) ? disclosed via caveat? Run repros. Report findings.

2. assistant (2026-07-07T00:17:12.076Z)

I'll review PR #57's /why command for input parsing and rendering robustness defects. Let me start by reading the actual code.

3. user (2026-07-07T00:17:14.871Z)

```
12b4442 feat(commands): add why explainability command
6149c38 feat(policy): persist policy decision traces (#55)
da0ab47 feat(policy): add deep inspection budget boundary (#54)
a7cbbda feat(policy): add URL reputation stage (#53)
5cd65d1 feat(policy): add caption safety stage (#52)
---DIFF---
src/tg_video_filter/command_service.py | 120 ++++++
src/tg_video_filter/commands.py        |   2 +
src/tg_video_filter/db.py              |  16 +++
tests/test_command_service.py          | 146 ++++++
tests/test_db.py                      |  13 +++
5 files changed, 295 insertions(+), 2 deletions(-)
```

4. user (2026-07-07T00:17:15.626Z)

```
./src/tg_video_filter/command_service.py
```

5. assistant (2026-07-07T00:17:17.422Z)

Let me read the actual diff of the source files.

6. user (2026-07-07T00:17:19.824Z)

```
diff --git a/src/tg_video_filter/command_service.py b/src/tg_video_filter/command_service.py
index 59b7d81..0e3a390 100644
--- a/src/tg_video_filter/command_service.py
+++ b/src/tg_video_filter/command_service.py
@@ -11,6 +11,7 @@ See ``docs/adr/0007-inline-config-commands.md`` for the command model and

from __future__ import annotations

+import json
import logging
import re
from dataclasses import dataclass
@@ -18,7 +19,8 @@ from dataclasses import dataclass
import aiosqlite

from tg_video_filter import db
-from tg_video_filter.models import FilterConfig, Source
+from tg_video_filter.filters import FilterEngine, VideoMeta
+from tg_video_filter.models import ContentItem, FilterConfig, PolicyDecision, Source

log = logging.getLogger(__name__)

@@ -33,6 +35,7 @@ _MIN_LENGTH_RE = re.compile(r"^/set\s+min\s+length\s+(\d+)\s*([smh]?)$",
re.IGNORECASE)
_MIN_RES_RE = re.compile(r"^/set\s+min\s+(?:resolution|res)\s+(\d+)\s*p?$", re.IGNORECASE)
_SHOW_CONFIG_RE = re.compile(r"^/show\s+config$", re.IGNORECASE)
_SHOW_COMMANDS_RE = re.compile(r"^/show\s+commands$", re.IGNORECASE)
+WHY_RE = re.compile(r"^/why(?:\s+(.*))?$", re.IGNORECASE)
_EXCLUDE_RE = re.compile(r"^/set\s+exclude\s+(gif|round|nosound)\s+(on|off)$", re.IGNORECASE)
_EXCLUDE_HELP_RE = re.compile(r"^/set\s+exclude\s+", re.IGNORECASE)
_MIN_RES_HELP_RE = re.compile(r"^/set\s+min\s+(?:resolution|res)\s+", re.IGNORECASE)
@@ -160,12 +163,97 @@ def format_commands() -> str:
    ``/unwatch <id|@name>` ? Stop watching a source",
    ``/watchlist` ? Show only watched sources",
```

```

        """
+       "? **Explainability**",
+       "`/why <content_item_id>` ? Explain why an item passed or failed",
+       """
+       "\u2139\u2013 **Help**",
+       "`/show commands` ? Show this help message",
    ]
)

+def _format_json_value(value: object, max_len: int = 500) -> str:
+    text = json.dumps(value, default=str, ensure_ascii=False, sort_keys=True)
+    if len(text) <= max_len:
+        return text
+    return f"{text[: max_len - 1]}?"

+def _content_item_to_video_meta(item: ContentItem) -> VideoMeta:
+    """Build a metadata replay input from fields persisted on content_items."""
+    return VideoMeta(
+        duration_s=item.duration_s,
+        width=item.width,
+        height=item.height,
+        size_bytes=item.size_bytes,
+        mime_type=item.mime_type,
+    )

+def _format_policy_decisions(item: ContentItem, decisions: list[PolicyDecision]) -> str:
+    status = "PASSED" if decisions[-1].passed else "REJECTED"
+    lines = [
+        f"? **Why content item `{item.id}`?",
+        """
+        f"Final policy result: **{status}**",
+        f"Provider message: `{item.provider}:{item.provider_message_id}`",
+    ]
+    if item.link_url:
+        lines.append(f"Link: {item.link_url}")
+    if item.text:
+        lines.append(f"Text: {_format_json_value(item.text, max_len=160)}")
+
+    for decision in decisions:
+        decision_status = "PASSED" if decision.passed else "REJECTED"
+        lines.extend(
+            [
+                """
+                f"Policy `{decision.policy_id}`: **{decision_status}**",
+                f"Evaluated: `{decision.evaluated_at.isoformat()}`",
+            ]
+        )
+    if not decision.trace:
+        lines.append("- No stage trace recorded.")
+        continue
+    for stage in decision.trace:
+        stage_name = str(stage.get("stage_name", "unknown"))
+        stage_passed = bool(stage.get("passed"))
+        reason = str(stage.get("reason", "no reason recorded"))
+        icon = "?" if stage_passed else ""
+        lines.append(f"- {icon} `{stage_name}` ? {reason}")
+        metadata = stage.get("metadata")
+        if isinstance(metadata, dict) and metadata:
+            lines.append(f"  metadata: {_format_json_value(metadata)}")
+    return "\n".join(lines)

```

```

+def _format_missing_policy_decision(item: ContentItem, current_cfg: FilterConfig) -> str:
+    recorded_status = "accepted" if item.matched else "rejected"
+    replay = FilterEngine(current_cfg).evaluate(_content_item_to_video_meta(item))
+    replay_status = "accepted" if replay.matched else "rejected"
+    lines = [
+        f"? **Why content item `{item.id}`?***",
+        "",
+        "No policy-stage decision trace was recorded for this item.",
+        f"Recorded item status: **{recorded_status}**.",
+        "",
+        "Current metadata-filter replay:",
+        f"- Result with current config: **{replay_status}**",
+    ]
+    if replay.reasons:
+        lines.extend(f"- {reason}" for reason in replay.reasons)
+    else:
+        lines.append("- No metadata filters are currently configured.")
+    lines.extend(
+        [
+            "",
+            "This fallback uses the current filter config and persisted metadata only; "
+            "historical config changes can make it differ from the original decision.",
+        ]
+    )
+    return "\n".join(lines)
+
+
+
+# --- command result -----
+
@@ -335,6 +423,11 @@ class CommandService:
+    if _WATCHLIST_RE.match(text):
+        return await self._handle_watchlist(conn, user_id)
+
+    # --- /why <content_item_id> -----
+    m = _WHY_RE.match(text)
+    if m:
+        return await self._handle_why(conn, user_id, m.group(1), current_cfg)
+
+    # --- /show commands -----
+    if _SHOW_COMMANDS_RE.match(text):
+        return CommandResult(format_commands())
@@ -405,6 +498,30 @@ class CommandService:
+    lines.append(f"? `{s.provider_source_id}` ? {label}")
+    return CommandResult("\n".join(lines))
+
+    async def _handle_why(
+        self,
+        conn: aiosqlite.Connection,
+        user_id: int,
+        item_id_text: str | None,
+        current_cfg: FilterConfig,
+    ) -> CommandResult:
+        """Explain a recorded content item's policy decision."""
+        if item_id_text is None or not item_id_text.strip().isdecimal():
+            return CommandResult("? Usage: `/why <content_item_id>`")
+        item_id = int(item_id_text.strip())
+        if item_id < 1:
+            return CommandResult("? Usage: `/why <content_item_id>`")
+
+        item = await db.get_content_item_for_account(conn, user_id, item_id)
+        if item is None:
+            return CommandResult(f"? Content item `{item_id}` was not found for your account.")
+
+        assert item.id is not None

```

```

+ decisions = await db.list_policy_decisions_for_content(conn, item.id)
+ if decisions:
+     return CommandResult(_format_policy_decisions(item, decisions))
+ return CommandResult(_format_missing_policy_decision(item, current_cfg))
+
+ async def _resolve_source(
+     self, conn: aiomysqlite.Connection, user_id: int, target: str
+ ) -> Source | None:
@@ -482,6 +599,7 @@ __all__ = [
+     "ENGINE_KEY",
+     "CommandResult",
+     "CommandService",
+     "_WHY_RE",
+     "_format_duration",
+     "_format_size",
+     "format_config",
diff --git a/src/tg_video_filter/commands.py b/src/tg_video_filter/commands.py
index 3aea1f6..06db189 100644
--- a/src/tg_video_filter/commands.py
+++ b/src/tg_video_filter/commands.py
@@ -41,6 +41,7 @@ from tg_video_filter.command_service import ( # noqa: E402, F401
+     _UNWATCH_RE,
+     _WATCH_RE,
+     _WATCHLIST_RE,
+     _WHY_RE,
+     _format_duration,
+     _format_size,
+     format_commands,
@@ -142,6 +143,7 @@ __all__ = [
+     "_EXCLUDE_HELP_RE",
+     "_MIN_RES_HELP_RE",
+     "_SIZE_RE",
+     "_WHY_RE",
+     "_format_duration",
+     "_format_size",
+     "_update_config",
diff --git a/src/tg_video_filter/db.py b/src/tg_video_filter/db.py
index 6cdd65f..0d1069d 100644
--- a/src/tg_video_filter/db.py
+++ b/src/tg_video_filter/db.py
@@ -1285,6 +1285,21 @@ async def get_content_item_by_dedup_key(
+     return _row_to_content_item(row) if row else None
+
+ async def get_content_item_for_account(
+     conn: aiomysqlite.Connection,
+     account_id: int,
+     content_item_id: int,
+ ) -> ContentItem | None:
+     """Read a content item by id, scoped to an account for user-facing lookups."""
+     conn.row_factory = aiomysqlite.Row
+     async with conn.execute(
+         "SELECT * FROM content_items WHERE id = ? AND account_id = ?",
+         (content_item_id, account_id),
+     ) as cur:
+         row = await cur.fetchone()
+     return _row_to_content_item(row) if row else None
+
+ # --- Policy repos (ADR 0011, Option 2A) -----
@@ -1973,6 +1988,7 @@ __all__ = [
+     "insert_content_item",
+     "get_content_item_by_message",

```

```

    "get_content_item_by_dedup_key",
+   "get_content_item_for_account",
    "upsert_policy",
    "get_default_policy",
    "load_policy_config",

```

7. assistant (2026-07-07T00:17:23.181Z)

Now let me focus on the key robustness dimensions. First the ``isdecimal()`` unicode issue, and the rendering. Let me check the ``_format_policy_decisions`` ? I notice ``item.link_url`` and ``item.text`` are echoed. Let me look at what fields exist and check the unicode digit path.

8. user (2026-07-07T00:17:27.706Z)

```

'???' -> isdecimal= True int= 123 err= None
'???' -> isdecimal= True int= 123 err= None
'2' -> isdecimal= False int= None err= None
'?' -> isdecimal= False int= None err= None
' 42 ' -> isdecimal= True int= 42 err= None
'4_2' -> isdecimal= False int= None err= None
'?' -> isdecimal= True int= 4 err= None
'0' -> isdecimal= True int= 0 err= None
'00' -> isdecimal= True int= 0 err= None
'1?3' -> isdecimal= True int= 123 err= None

```

9. assistant (2026-07-07T00:17:32.488Z)

So the ``isdecimal()`` + ``int()`` combination is safe ? Python's ``int()`` accepts every character that ``isdecimal()`` accepts (superscripts are ``isdigit`` but not ``isdecimal``, so they're correctly rejected). No crash there. The ``int()`` guard is robust. Now let me look at the rendering side ? the markdown injection and message-size concerns. Let me check what `parse_mode` is used and how ``_reply`` sends, and what ``item.link_url`/`item.text`` contain.

10. user (2026-07-07T00:17:38.223Z)

```

src/tg_video_filter/commands.py:24:from tg_video_filter.command_service import ENGINE_KEY,
CommandResult, CommandService
src/tg_video_filter/commands.py:80:         result: CommandResult = await svc.handle(user.id,
text, conn, current_cfg)
src/tg_video_filter/commands.py:110:         :meth:`CommandService.handle` which returns a
:class:`CommandResult`.
src/tg_video_filter/commands.py:128:async def _reply(client: TelegramClient, to_message: object,
text: str) -> None:
src/tg_video_filter/command_service.py:5:front-door. The service is pure: strings in,
`CommandResult` out ? no
src/tg_video_filter/command_service.py:261:class CommandResult:
src/tg_video_filter/command_service.py:279:         returning a ``CommandResult``. No Telethon
objects involved ? pure string
src/tg_video_filter/command_service.py:301:         ) -> CommandResult:
src/tg_video_filter/command_service.py:315:         return CommandResult("? Invalid size
format. Use e.g. `/set size 10MB`.")
src/tg_video_filter/command_service.py:317:         return CommandResult("? Size must be
at least 1 MB (`FilterConfig` constraint).")
src/tg_video_filter/command_service.py:319:         return CommandResult(
src/tg_video_filter/command_service.py:331:         return CommandResult("? Invalid
duration format. Use e.g. `/set length 15m`.")
src/tg_video_filter/command_service.py:334:         return CommandResult(
src/tg_video_filter/command_service.py:339:         return CommandResult("? Invalid
duration format. Use e.g. `/set min length 5s`.")
src/tg_video_filter/command_service.py:354:         return CommandResult(
src/tg_video_filter/command_service.py:359:         return CommandResult(
src/tg_video_filter/command_service.py:369:         return CommandResult("? Resolution
must be at least 1p.")

```

```

src/tg_video_filter/command_service.py:371:         return CommandResult(
src/tg_video_filter/command_service.py:377:         return CommandResult(
src/tg_video_filter/command_service.py:394:         return CommandResult(
src/tg_video_filter/command_service.py:401:         return CommandResult(
src/tg_video_filter/command_service.py:408:         return
src/tg_video_filter/command_service.py:415:         CommandResult(format_config(current_cfg))
src/tg_video_filter/command_service.py:433:         return CommandResult(format_commands())
src/tg_video_filter/command_service.py:436:         return CommandResult()
src/tg_video_filter/command_service.py:442:     ) -> CommandResult:
src/tg_video_filter/command_service.py:446:         return CommandResult(
src/tg_video_filter/command_service.py:454:         return CommandResult(f"? Now watching
src/tg_video_filter/command_service.py:458:     ) -> CommandResult:
src/tg_video_filter/command_service.py:462:         return CommandResult(
src/tg_video_filter/command_service.py:467:         return CommandResult(f"? Stopped watching
src/tg_video_filter/command_service.py:469:     async def _handle_channels(self, conn:
aiosqlite.Connection, user_id: int) -> CommandResult:
src/tg_video_filter/command_service.py:473:         return CommandResult(
src/tg_video_filter/command_service.py:484:         return CommandResult("\n".join(lines))
src/tg_video_filter/command_service.py:486:     async def _handle_watchlist(self, conn:
aiosqlite.Connection, user_id: int) -> CommandResult:
src/tg_video_filter/command_service.py:490:         return CommandResult(
src/tg_video_filter/command_service.py:499:         return CommandResult("\n".join(lines))
src/tg_video_filter/command_service.py:507:     ) -> CommandResult:
src/tg_video_filter/command_service.py:510:         return CommandResult("? Usage: `/why
src/tg_video_filter/command_service.py:513:         return CommandResult("? Usage: `/why
src/tg_video_filter/command_service.py:515:         <content_item_id>")
src/tg_video_filter/command_service.py:518:         <content_item_id>")

```

11. user (2026-07-07T00:17:41.000Z)

```

128     async def _reply(client: TelegramClient, to_message: object, text: str) -> None:
129         try:
130             await client.send_message("me", text)
131         except (OSError, TimeoutError, RPCError):
132             log.warning("config_reply_failed", exc_info=True)
133
134
135     __all__ = [
136         "ENGINE_KEY",
137         "_EXCLUDE_RE",
138         "_LENGTH_RE",
139         "_MIN_LENGTH_RE",
140         "_MIN_RES_RE",
141         "_SHOW_CONFIG_RE",
142         "_SHOW_COMMANDS_RE",
143         "_EXCLUDE_HELP_RE",
144         "_MIN_RES_HELP_RE",
145         "_SIZE_RE",
146         "_WHY_RE",
147         "_format_duration",
148         "_format_size",
149         "_update_config",
150         "format_commands",
151         "format_config",
152         "make_config_command_handler",
153         "parse_duration_s",
154         "parse_size_mb",
155     ]
156

```

12. assistant (2026-07-07T00:17:44.756Z)

Now the key question: what `parse\_mode` does `send\_message` use? Telethon's default

`send\_message` parses markdown by default. Let me confirm and check the content fields echoed (`link\_url`, `text`, `provider\_message\_id`, stage `reason`, `metadata`).

13. user (2026-07-07T00:17:46.898Z)

```
23:     """How a matched video is delivered to the user.
149:class ContentItem(BaseModel):
153:     (`content_type`='video'); `text` / `link_url` are landed now but stay
165:     provider_message_id: str = Field(
169:     text: str | None = Field(default=None, description="Caption / post text (nullable in
Phase 1)")
170:     link_url: str | None = Field(default=None, description="For link/article posts (Phase 1:
NULL)")
182:     matched: bool = False
222:     """A delivery target ? where matched content is sent (ADR 0011, Track C).
352:     # How matched videos are delivered: forward the video, send a link, or both.
357:     # Where matched videos / link messages are sent. Accepts:
413:     text: str | None = Field(default=None, description="Caption / message text")
426:     matched: bool = False
```

14. user (2026-07-07T00:17:47.827Z)

```
src/tg_video_filter/backfill.py:365:         await client.send_message("me", text)
src/tg_video_filter/commands.py:130:         await client.send_message("me", text)
src/tg_video_filter/delivery.py:177:     ok = await _send_with_retry(client, lambda:
client.send_message(target, text), video_id)
src/tg_video_filter/models.py:362:     # The string is passed verbatim to Telethon's
forward_messages / send_message.
Binary file src/tg_video_filter/__pycache__/backfill.cpython-313.pyc matches
Binary file src/tg_video_filter/__pycache__/commands.cpython-313.pyc matches
Binary file src/tg_video_filter/__pycache__/delivery.cpython-313.pyc matches
```

15. assistant (2026-07-07T00:17:54.266Z)

`send\_message("me", text)` with no `parse\_mode` argument ? Telethon defaults to markdown parsing when the client's default parse_mode is set (the app sets it, or Telethon's global default). The messages are clearly written for markdown (backticks, `\*\*bold\*\*`). So markdown IS being parsed. This is the crux of the injection concern.

`item.text` (caption) and `item.link\_url` are attacker-controlled (they come from the source channel's post). Let me verify how they're rendered and whether an unbalanced backtick or markdown breaks the parse. Let me check the exact rendering lines again and test.

16. user (2026-07-07T00:17:54.993Z)

```
189         mime_type=item.mime_type,
190     )
191
192
193     def _format_policy_decisions(item: ContentItem, decisions: list[PolicyDecision]) -> str:
194         status = "PASSED" if decisions[-1].passed else "REJECTED"
195         lines = [
196             f"? **Why content item `{item.id}`?**",
197             "",
198             f"Final policy result: **{status}**",
199             f"Provider message: `{item.provider}:{item.provider_message_id}`",
200         ]
201         if item.link_url:
202             lines.append(f"Link: {item.link_url}")
203         if item.text:
204             lines.append(f"Text: {_format_json_value(item.text, max_len=160)}")
205
206         for decision in decisions:
207             decision_status = "PASSED" if decision.passed else "REJECTED"
```



```

208         lines.extend(
209             [
210                 "",
211                 f"Policy `{decision.policy_id}`: **{decision_status}**",
212                 f"Evaluated: `{decision.evaluated_at.isoformat()}`",
213             ]
214         )
215         if not decision.trace:
216             lines.append("- No stage trace recorded.")
217             continue
218         for stage in decision.trace:
219             stage_name = str(stage.get("stage_name", "unknown"))
220             stage_passed = bool(stage.get("passed"))
221             reason = str(stage.get("reason", "no reason recorded"))
222             icon = "?" if stage_passed else "?"
223             lines.append(f"- {icon} `{stage_name}` ? {reason}")
224             metadata = stage.get("metadata")
225             if isinstance(metadata, dict) and metadata:
226                 lines.append(f"    metadata: `{_format_json_value(metadata)}`")
227         return "\n".join(lines)
228
229
230     def _format_missing_policy_decision(item: ContentItem, current_cfg: FilterConfig) ->
str:
231         recorded_status = "accepted" if item.matched else "rejected"
232         replay = FilterEngine(current_cfg).evaluate(_content_item_to_video_meta(item))
233         replay_status = "accepted" if replay.matched else "rejected"

```

17. assistant (2026-07-07T00:18:01.182Z)

Now I can see the injection surfaces clearly. Line 199, 202, 204, 211, 212, 223, 226 embed values into a markdown-parsed message. The attacker-controlled ones are:

- Line 202: `Link: {item.link\_url}` ? raw, no backticks, no escaping
- Line 204: `Text: {\_format\_json\_value(item.text)}` ? JSON-encoded so quotes are escaped, but backticks/asterisks survive
- Line 223: `reason` inside stage trace (may be attacker-influenced)
- Line 199: `provider\_message\_id` inside backticks

Let me test what Telethon's markdown parser does with unbalanced backticks / an entity that overflows the message. Let me test the actual failure: an unclosed backtick from `provider\_message\_id` or a `link\_url` containing markdown. Let me confirm Telethon is available and reproduce.

18. user (2026-07-07T00:18:07.796Z)

```

LINK CASE stripped text: 'Link: http://evil.com) pwn `code`'
entities: [<telethon.tl.types.MessageEntityTextUrl object at 0x00000192D1077770>]
---
PMID CASE stripped: 'Provider message: telegram:msg123`nFinal policy result: REJECTED'
entities: [<telethon.tl.types.MessageEntityCode object at 0x00000192D10778C0>,
<telethon.tl.types.MessageEntityBold object at 0x00000192D1077A10>]

```

19. assistant (2026-07-07T00:18:17.270Z)

This confirms markdown injection. In the LINK CASE, the attacker's `[pwn](http://x)` became a real clickable `MessageEntityTextUrl` link entity injected into what should be an explainability report. The `link\_url` is rendered raw (line 202) with no escaping, so any markdown in a caption/link renders as active formatting ? including injecting arbitrary clickable hyperlinks (`[label](url)`) into the bot's own trusted output. This is a markdown injection / content-spoofing finding.

Now let me check the more serious failure mode: does an *unbalanced/mangled* entity cause the **send to fail entirely** (RPCError, swallowed by `\_reply`), meaning the user gets NOTHING back? That would be a functional break, not just cosmetic. Let me test the message-length concern and

whether Telethon rejects malformed entities.

20. user (2026-07-07T00:18:21.298Z)

```
json text field: '""'
parsed: 'Text: '""' []
---MSG LENGTH---
len parsed: 5000
```

21. assistant (2026-07-07T00:18:26.861Z)

A single unbalanced `` is left as literal (Telethon only forms an entity on a matched pair). So an unbalanced-delimiter `*send failure*` is not the primary risk; the injection (well-formed attacker markdown ? active entities/links) is the real one, plus message length.

Now let me confirm the message-length concern is real. 5000-char message exceeds Telegram's 4096 limit ? ``send_message`` raises `RPCError` ? ``_reply`` swallows it ? user gets **no response at all** to ``/why``. Let me verify how many decisions/stages/metadata can realistically stack, and that there's no cap in the formatter. Let me look at whether decisions accumulate (multiple policies over an item's life) and metadata size.

22. user (2026-07-07T00:18:28.810Z)

```
async def list_policy_decisions_for_content(
    conn: aiomysqlite.Connection,
    content_item_id: int,
) -> list[PolicyDecision]:
    """Return all policy decisions recorded for a content item."""
    conn.row_factory = aiomysqlite.Row
    async with conn.execute(
        """SELECT * FROM policy_decisions
           WHERE content_item_id = ?
           ORDER BY evaluated_at, id""",
        (content_item_id,),
    ) as cur:
        rows = await cur.fetchall()
    return [_row_to_policy_decision(row) for row in rows]
```

23. assistant (2026-07-07T00:18:39.592Z)

All decisions are returned unbounded, each with a full trace (multiple stages), each stage's ``metadata`` truncated to 500 chars but the **number** of stages and decisions is unbounded, plus ``reason`` (line 223) is **not** truncated at all. So the total message is unbounded. Let me quantify a realistic overflow and confirm reason is untruncated.

Line 221: ``reason = str(stage.get("reason", ...))`` ? rendered raw on line 223 with no length cap. If any stage's reason is long (e.g., an error message, a long caption echoed into a reason), a single stage line can be huge. Combined with unbounded decisions/stages, the message easily exceeds 4096.

Let me verify the practical impact: construct a realistic content item that produces a >4096 message and confirm the actual code path builds it, then confirm send would fail. Let me build an integration-style repro against the real formatter.

24. user (2026-07-07T00:18:42.022Z)

```
249:     ``policy_id`` is NULL, the account's default policy is used. When multiple
258:     policy_id: int | None = Field(
299: class PolicyDecision(BaseModel):
300:     """Persisted ADR 0012 policy evaluation trace for one content item."""
306:     policy_id: int = Field(description="FK -> policies.id")
307:     passed: bool
--
310:         description="Ordered StageResult trace serialized in policy_decisions.trace_json",
```

```

312:     evaluated_at: datetime = Field(default_factory=_utcnow)
---
class PolicyDecision(BaseModel):
    """Persisted ADR 0012 policy evaluation trace for one content item."""

    model_config = ConfigDict(frozen=True)

    id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
    content_item_id: int = Field(description="FK -> content_items.id")
    policy_id: int = Field(description="FK -> policies.id")
    passed: bool
    trace: list[dict[str, object]] = Field(
        default_factory=list,
        description="Ordered StageResult trace serialized in policy_decisions.trace_json",
    )
    evaluated_at: datetime = Field(default_factory=_utcnow)

```

```
class FilterConfig(BaseModel):
```

25. user (2026-07-07T00:18:52.630Z)

```

Traceback (most recent call last):
  File "<string>", line 16, in <module>
    item = ContentItem(id=1, account_id=1, provider='telegram', provider_source_id='s',
provider_message_id='m', content_type='video')
  File "C:\Users\avidu\Projects\Telegram\.venv\Lib\site-packages\pydantic\main.py", line 263, in
__init__
    validated_self = self.__pydantic_validator__.validate_python(data, self_instance=self)
pydantic_core._pydantic_core.ValidationError: 2 validation errors for ContentItem
source_id
  Field required [type=missing, input_value={'id': 1, 'account_id': 1...'content_type':
'video'}, input_type=dict]
  For further information visit https://errors.pydantic.dev/2.13/v/missing
dedup_key
  Field required [type=missing, input_value={'id': 1, 'account_id': 1...'content_type':
'video'}, input_type=dict]
  For further information visit https://errors.pydantic.dev/2.13/v/missing

```

26. user (2026-07-07T00:19:00.555Z)

```

MESSAGE LENGTH: 17781 EXCEEDS 4096: True
one reason len (untruncated): 627

```

27. assistant (2026-07-07T00:19:07.534Z)

Confirmed: a realistic `/why` output reaches 17,781 chars ? over 4x Telegram's 4096 limit. Telethon's `send_message` will raise `MessageTooLongError` (an `RPCError` subclass), which `_reply` (commands.py:131) silently swallows ? the user gets **zero response** to their `/why` command. The item that most needs explaining (many stages, long reasons) is exactly the one that produces no answer.

Note the asymmetry the author was aware of: `metadata` is truncated to 500 (line 226), and `item.text` to 160 (line 204), but `reason` (line 223) is **not truncated at all**, and the overall message has **no cap** regardless of per-value truncation.

Let me now verify the `link_url` injection concern is fully real by confirming `link_url`/`text` are populated from attacker-controlled source posts (not always NULL in this phase). The model comment said "Phase 1: NULL" for `link_url`. Let me check whether these are actually populated yet ? if always NULL, the injection is latent, not live.

28. user (2026-07-07T00:19:09.697Z)

```
src/tg_video_filter/backfill.py:243: text = _message_text(message)
```

```

src/tg_video_filter/backfill.py:273:         text=text,
src/tg_video_filter/backfill.py:302:         link_text = build_link_text(
src/tg_video_filter/backfill.py:315:             link_text=link_text,
src/tg_video_filter/backfill.py:383:         text = getattr(message, "message", "") or ""
src/tg_video_filter/command_service.py:176:         text = json.dumps(value, default=str,
ensure_ascii=False, sort_keys=True)
src/tg_video_filter/command_service.py:303:         text = text.rstrip()
src/tg_video_filter/db.py:898:         text=row["text"],
src/tg_video_filter/db.py:899:         link_url=row["link_url"],
src/tg_video_filter/db.py:1708:         text=row["text"],
src/tg_video_filter/db.py:1813:         text=video.text,
src/tg_video_filter/db.py:1833:         item = await insert_content_item(conn, item)
src/tg_video_filter/db.py:1867:         text=item.text,
src/tg_video_filter/db.py:1951:         text=r["text"],
src/tg_video_filter/db.py:1988:         "insert_content_item",
src/tg_video_filter/policy.py:365:         text = item.strip()
src/tg_video_filter/policy.py:441:         text = value.strip()
src/tg_video_filter/telethon_client.py:279:         text = _message_text(message)
src/tg_video_filter/telethon_client.py:309:         text=text,
src/tg_video_filter/telethon_client.py:355:         link_text = build_link_text(

```

29. assistant (2026-07-07T00:19:14.150Z)

`text` is populated from the real message caption (`\_message\_text(message)` at backfill.py:243, telethon_client.py:279) and persisted (db.py:1813, 1867). So `item.text` IS live attacker-controlled data (the caption of any post in a watched channel). It gets rendered into the markdown-parsed /why reply at line 204. The `link\_url` may still be NULL in this phase, but `text` is not ? the injection is live via captions.

Let me confirm the caption-injection end-to-end through the markdown parser with a realistic caption containing an injected link.

30. user (2026-07-07T00:19:19.693Z)

```

rendered line: Text: "Nice video [CLICK FOR PRIZE](http://phish.evil) and *totally passed*"
final visible text: 'Text: "Nice video CLICK FOR PRIZE and *totally passed*"'
injected entities: ['MessageEntityTextUrl']

```

31. assistant (2026-07-07T00:19:26.376Z)

Confirmed live: an attacker's caption `[CLICK FOR PRIZE](http://phish.evil)` renders as a real clickable hyperlink entity (`MessageEntityTextUrl`) inside the bot's trusted /why report. The JSON-encoding at line 176 escapes quotes/backslashes but does **not** neutralize markdown metacharacters (`[, ]`, `(`, `)`, `\*`, `\_`, `` ` ``). Note ***totally passed*** did NOT render bold here only because the closing `` ` `` breaks the pair ? but `[...](...)` link syntax needs no such balancing and injects cleanly.

Now let me double-check the `evaluated\_at.isoformat()` (line 212) and the metadata-replay fallback caveat concern, and whether the missing-decision fallback can crash on None metadata fields. Let me check `VideoMeta` required fields vs nullable content_item fields.

32. user (2026-07-07T00:19:28.840Z)

```

grep: src/tg_video_filter/filters.py: No such file or directory
---CONTENTITEM NULLABILITY---
class ContentItem(BaseModel):
    """A normalized content record discovered from any :class:`Source` (ADR 0011).

    Generalizes the legacy :class:`VideoRecord`. Phase 1 stores only videos
    (`content_type='video'`); `text` / `link_url` are landed now but stay
    NULL until later tracks populate them (avoids a future `ALTER TABLE`).
    `dedup_key` is the provider-neutral dedup key (Option 1A: tiered lookup
    wrapping ADR 0006, computed by :func:`tg_video_filter.db.compute_dedup_key`).
    """

```

```

model_config = ConfigDict(frozen=True)

id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
account_id: int = Field(description="FK ? accounts.id")
source_id: int = Field(description="FK ? sources.id")
provider: str = Field(default="telegram", description="Provider namespace")
provider_message_id: str = Field(
    description="Provider-native message id (Telegram message_id, ?)"
)
content_type: str = Field(default="video", description="'video' | 'link' | 'article'")
text: str | None = Field(default=None, description="Caption / post text (nullable in Phase
1)")
link_url: str | None = Field(default=None, description="For link/article posts (Phase 1:
NULL)")
media_ref: str | None = Field(default=None, description="Provider media reference")
document_id: str | None = Field(
    default=None,
    description="Telegram document.id (kept for TG-specific Tier-1 lookups)",
)
duration_s: int | None = None
width: int | None = None
height: int | None = None
size_bytes: int | None = None
mime_type: str | None = None
dedup_key: str = Field(description="Provider-neutral dedup key (Option 1A); unique per
account")
matched: bool = False
seen_at: datetime = Field(default_factory=_utcnow)

```

33. assistant (2026-07-07T00:19:32.116Z)

`duration\_s`, `width`, `height`, `size\_bytes`, `mime\_type` are all nullable (`int | None`). The fallback `\_content\_item\_to\_video\_meta` passes these straight into `VideoMeta`. Let me find the filters module and check whether `VideoMeta` accepts None and whether `FilterEngine.evaluate` crashes on None duration (e.g. a numeric comparison `None < min`).

34. user (2026-07-07T00:19:35.027Z)

```

src/tg_video_filter/filters/base.py
src/tg_video_filter/filters/__init__.py
---
src/tg_video_filter/filters/__init__.py

```

35. user (2026-07-07T00:19:42.241Z)

```

15:class VideoMeta(BaseModel):
23:    duration_s: int | None = None
24:    width: int | None = None
25:    height: int | None = None
26:    size_bytes: int | None = None
27:    mime_type: str | None = None
58:    A rule that cannot be evaluated (e.g. `duration_s is None`) must return
=== evaluate / min comparisons ===
5:whose bound is `None`), then evaluates a :class:`VideoMeta` against every
31:    def __init__(self, cfg: FilterConfig) -> None:
35:    def evaluate(self, meta: VideoMeta) -> FilterResult:
55:def _build_rules(cfg: FilterConfig) -> list[Filter]:
56:    """Build the rule list, skipping rules whose bound is None.
58:    (ResolutionFilter/SizeFilter also defend against None internally, but
63:    if cfg.min_duration_s is not None or cfg.max_duration_s is not None:
64:        rules.append(DurationFilter(cfg.min_duration_s, cfg.max_duration_s))
65:    if cfg.target_height is not None:
67:    if cfg.max_size_mb is not None:

```

36. assistant (2026-07-07T00:19:52.125Z)

`VideoMeta` fields are nullable and filters explicitly defend against `None` ("must return... a rule that cannot be evaluated"). So the fallback replay won't crash on None metadata ? the filters handle it. Good, no crash there.

I've now covered all the dimensions in the brief. Let me quickly verify the caveat text adequately discloses the content-type-filter inaccuracy (gif/round/nosound not persisted), which the brief flagged as a low finding.

37. user (2026-07-07T00:19:52.842Z)

```
230     def _format_missing_policy_decision(item: ContentItem, current_cfg: FilterConfig) ->
str:
231         recorded_status = "accepted" if item.matched else "rejected"
232         replay = FilterEngine(current_cfg).evaluate(_content_item_to_video_meta(item))
233         replay_status = "accepted" if replay.matched else "rejected"
234         lines = [
235             f"? **Why content item `{item.id}`?***",
236             "",
237             "No policy-stage decision trace was recorded for this item.",
238             f"Recorded item status: **{recorded_status}**.",
239             "",
240             "Current metadata-filter replay:",
241             f"- Result with current config: **{replay_status}**",
242         ]
243         if replay.reasons:
244             lines.extend(f"- {reason}" for reason in replay.reasons)
245         else:
246             lines.append("- No metadata filters are currently configured.")
247         lines.extend(
248             [
249                 "",
250                 "This fallback uses the current filter config and persisted metadata only; "
251                 "historical config changes can make it differ from the original decision.",
252             ]
253         )
254         return "\n".join(lines)
255
256
257     # --- command result -----
258
259
```

38. assistant (2026-07-07T00:20:02.818Z)

The caveat (lines 250-251) discloses "historical config changes can make it differ" but does NOT disclose that content-type exclusion filters (gif/round/nosound) aren't captured in the replay at all because `VideoMeta` only carries duration/width/height/size/mime. If the current config has `exclude gif on` but the replayed item was a gif, the replay's `matched` will be wrong (it can't see gif-ness), and the caveat doesn't mention this class of inaccuracy. This is a low-severity disclosure gap.

I have enough to report. Let me finalize the findings, ranked.

The findings:

1. **Medium** ? Oversized /why reply exceeds Telegram's 4096-char limit ? `MessageTooLongError` swallowed by `\_reply` ? user gets no response. `reason` is untruncated (line 223) and the whole message is uncapped. Reproduced at 17,781 chars.
2. **Medium** ? Markdown injection via attacker-controlled caption (`item.text`, line 204) and `link\_url` (line 202): `[label](url)` renders as a real clickable link entity inside the bot's trusted report. JSON-encoding doesn't neutralize markdown metacharacters.
3. **Low** ? Metadata-replay fallback caveat doesn't disclose that content-type filters (gif/round/nosound) are invisible to the replay, so `replay\_status` can be silently wrong for those items.

Note explicitly ruled out: the ``isdecimal()'+`int()``` guard is safe against unicode digits (Arabic-Indic/full-width parse identically to `int`; superscripts are rejected); the fallback replay does not crash on NULL metadata (filters defend against None).

39. user (2026-07-07T00:20:30.101Z)

Structured output provided successfully